Cloud-Native DevOps Platform

PART-5

SEETHA LAKSHMI K A P

29-06-2026

Serverless Automation using AWS Lambda and Amazon S3

1)Objective

To implement a serverless event-driven workflow using **AWS Lambda** and **Amazon S3**, enabling automatic execution of backend tasks whenever a new file is uploaded to the S3 bucket.

1. Lambda Project Files

1.1) lambda_function.py

Location:

`lambda/lambda_function.py`

Purpose:

Implements the AWS Lambda function that is automatically triggered whenever a new file is uploaded to the configured Amazon S3 bucket.

Important Functionalities:

- Receives the S3 Object Created event.
- Extracts the uploaded bucket name and file name.
- Reads sender and receiver email addresses from Lambda environment variables.
- Sends an email notification using Amazon SES.
- Records execution details in CloudWatch Logs.
- Returns a success or failure response after execution.

1.2) requirements.txt

Location:

`lambda/requirements.txt`

Purpose:

Specifies the Python dependency required by the Lambda function.

Dependency:

- boto3 (AWS SDK for Python)

1.3) lambda.zip

Location:

`lambda/lambda.zip`

Purpose:

Deployment package uploaded to AWS Lambda.

Contents:

- `lambda_function.py`

How it was Created:

Terraform automatically creates the ZIP package using the **archive_file** data source.

```
data "archive_file" "lambda_zip" {
  type        = "zip"
  source_file = "../lambda/lambda_function.py"
  output_path = "../lambda/lambda.zip"
}
```

Reason:

AWS Lambda requires the application code to be uploaded as a ZIP deployment package. Terraform automatically generates this package during deployment, eliminating the need to create it manually.

2. Terraform Files Used

2.1) lambda_iam.tf

Purpose:

Creates the IAM Role and IAM Policy required for the Lambda function.

Permissions Configured:

- CloudWatch Logs
- Amazon SES
- Lambda Basic Execution

Reason:

Allows the Lambda function to:

- Write execution logs to CloudWatch.
- Send email notifications using Amazon SES.

2.2) s3_notification.tf

Purpose:

Automates the complete deployment of the serverless workflow.

Resources Created:

- Generates **lambda.zip** automatically.
- Creates the AWS Lambda function.
- Configures Lambda environment variables.
- Grants Amazon S3 permission to invoke Lambda.
- Creates the S3 Event Notification.

Reason:

Automates the complete Lambda deployment using Infrastructure as Code (Terraform), eliminating manual configuration.

3. Deploy Serverless Resources

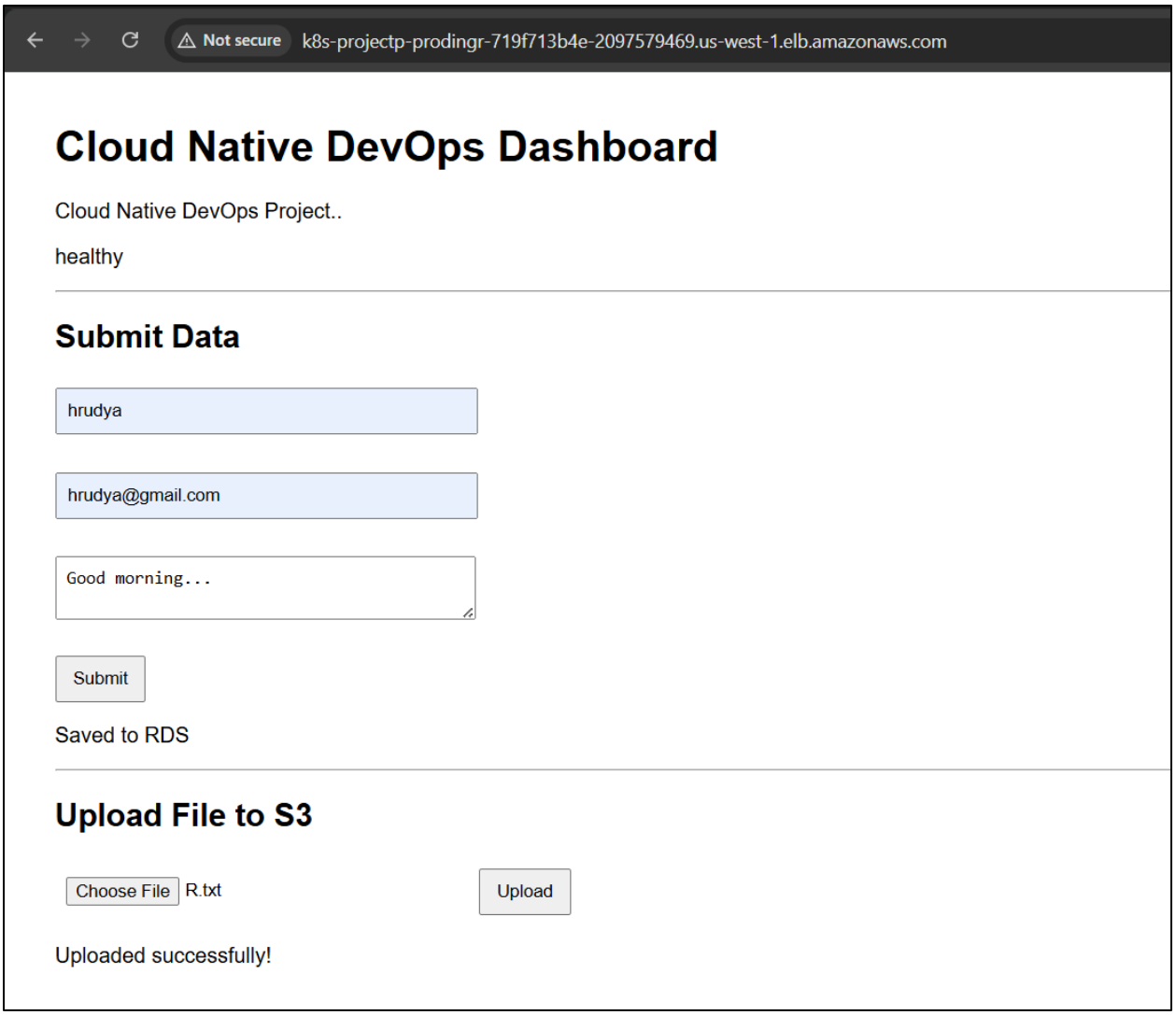
All Lambda resources were created automatically using Terraform.

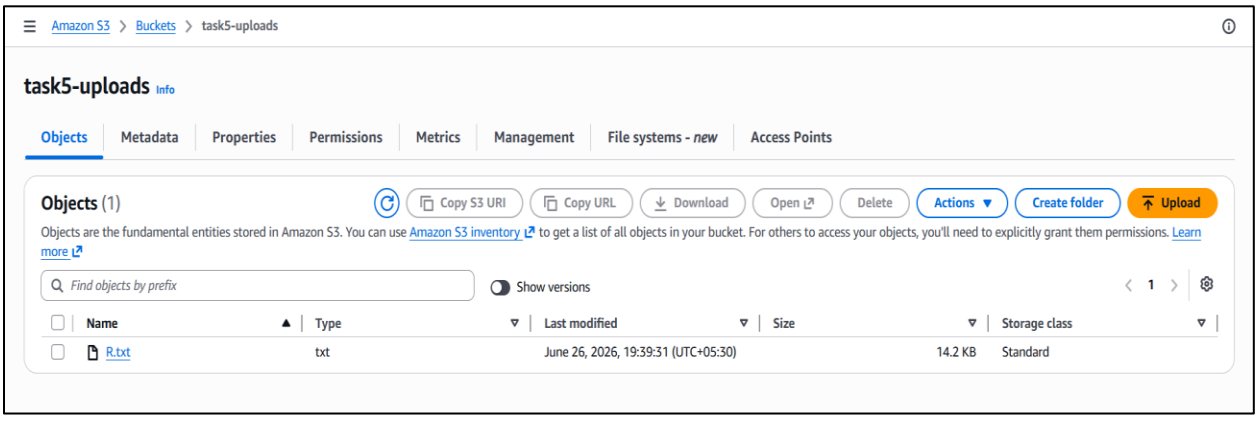
Command Used:

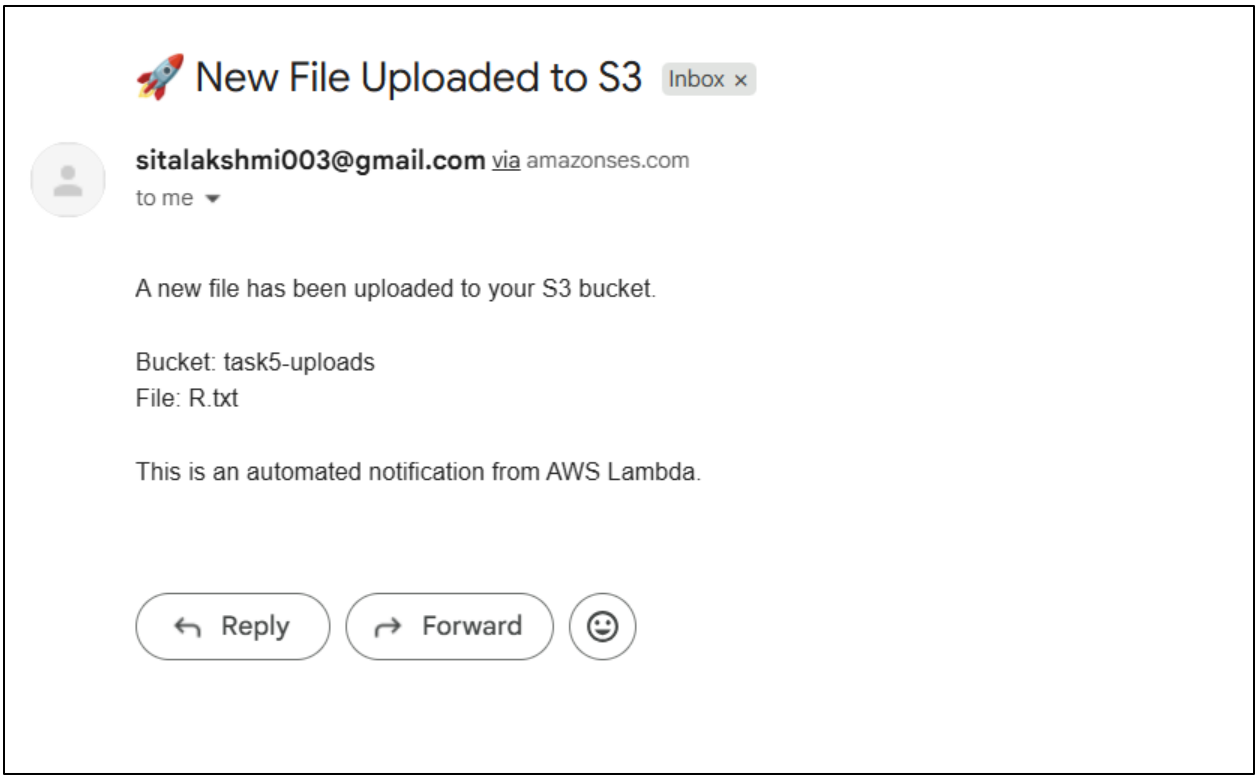
```
terraform apply
```

Resources Created:

- Lambda Deployment Package
- Lambda Function
- IAM Role
- IAM Policy
- Lambda Invoke Permission
- Environment Variables
- S3 Event Notification


Not secure
k8s-projectp-prodingr-719f713b4e-2097579469.us-west-1.elb.amazonaws.com
Cloud Native DevOps Dashboard
Cloud Native DevOps Project..
healthy
Submit Data
hrudya
hrudya@gmail.com
Good morning...
Submit
Saved to RDS
Upload File to S3
Choose File
R.txt
Upload
Uploaded successfully!


Amazon S3
Buckets
task5-uploads
task5-uploads
Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems - new
Access Points
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Show versions
Name
Type
Last modified
Size
Storage class
R.txt
txt
June 26, 2026, 19:39:31 (UTC+05:30)
14.2 KB
Standard



4. Amazon RDS Database Integration

The backend application was integrated with an **Amazon RDS MySQL** database to store user information submitted through the application.

Database Details

- Database Name: **appdb**
- Database Engine: **Amazon RDS MySQL**

Purpose

- Store application data persistently.
- Enable secure access from the backend application running inside the EKS cluster.
- Avoid data loss when application containers restart.

4.1) Accessing the Amazon RDS Database

Since the RDS instance is deployed in a private subnet, it was accessed through the backend pod running inside the EKS cluster.

Step 1: List Backend Pods

```
kubectl get pods -n project-prod
```

Step 2: Access the Backend Pod

```
kubectl exec -it <backend-pod-name> -n project-prod -- bash
```

Step 3: Open the Python Interpreter

```
python
```

4.2) Connect to the Database

The backend project already contains the **database.py** file, which establishes the connection to the Amazon RDS database.

Inside the Python interpreter, execute:

```
from app import get_connection

conn = get_connection()
cursor = conn.cursor()
```

Purpose

- Import the database connection function.
- Establish a connection with the Amazon RDS database.
- Create a cursor to execute SQL queries.

4.3) Create the Users Table

Execute the SQL command to create the application table:

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
)
""")

conn.commit()
```

Purpose

- Create the **users** table inside the **appdb** database.
- `IF NOT EXISTS` prevents duplicate table creation if it already exists.

4.4) Verify Table Creation

Execute the following command:

```
cursor.execute("SHOW TABLES")
print(cursor.fetchall())
```

Purpose

Verify that the **users** table has been created successfully.

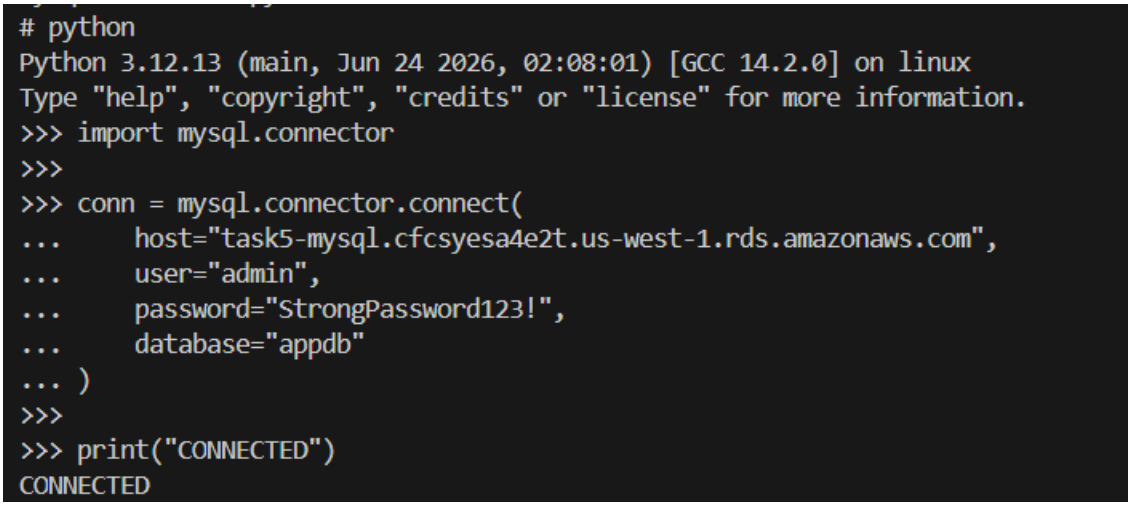
4.5) Verify Stored Data

To view the user records stored by the backend application, execute:

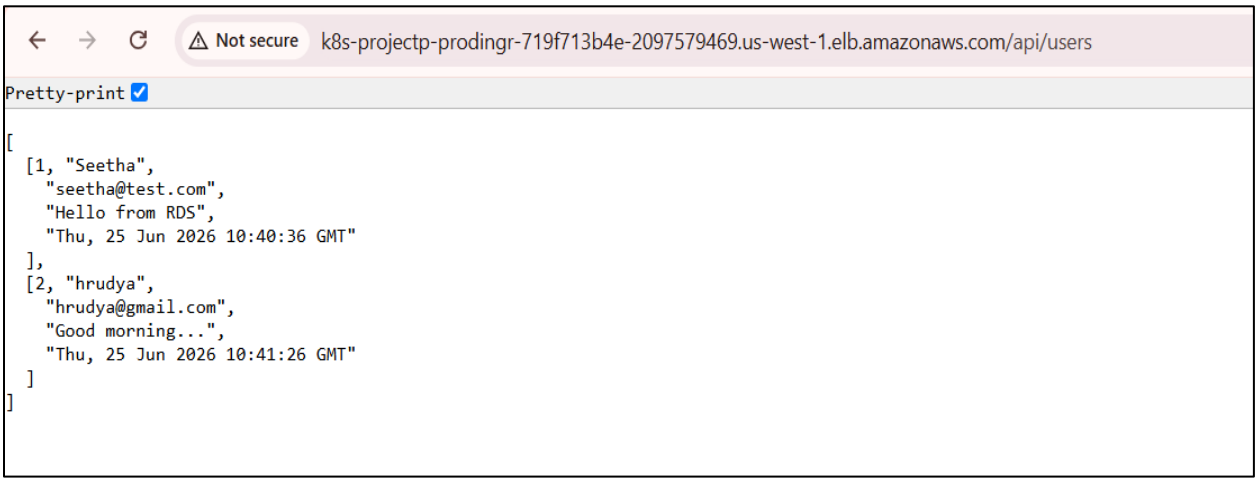
```
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())
```

Purpose

- Display all records stored in the **users** table.
- Verify that data submitted through the frontend application is successfully stored in the Amazon RDS database.



```
>>> cursor.execute("SELECT DATABASE()")
>>> print(cursor.fetchall())
[('appdb',)]
>>> cursor.execute("SHOW TABLES")
>>> print(cursor.fetchall())
[('users',)]
>>> cursor.execute("SELECT * FROM users")
>>> print(cursor.fetchall())
[(1, 'Seetha', 'seetha@test.com', 'Hello from RDS', datetime.datetime(2026, 6, 25, 10, 40, 36)), (2, 'hrudya', 'hrudya@gmail.com', 'Good morning...', dat
etime.datetime(2026, 6, 25, 10, 41, 26))]
>>>
```



5. Final Project Workflow

1. Provision the complete AWS infrastructure using **Terraform**, including VPC, EKS, RDS, ECR, S3, IAM Roles, Lambda, and supporting resources.
2. Develop the frontend and backend applications and containerize them using **Docker**.
3. Push the source code to the **GitHub** repository.
4. **GitHub Actions (CI)** automatically triggers on every code push to:
    a. Build the application.
    b. Run automated tests.
    c. Build Docker images.
    d. Push images to **Amazon ECR**.
    e. Update the Kubernetes manifests with the latest image tag.
5. **ArgoCD (CD)** continuously monitors the Git repository and synchronizes the updated Kubernetes manifests to the **Amazon EKS** cluster.
6. **Argo Rollouts** performs **Canary Deployment** for the frontend and backend applications, enabling controlled application updates with rollback support.
7. The application is exposed externally through the **AWS Load Balancer Controller** and **Ingress**, allowing users to access the application.
8. The backend application processes user requests by:
    a. Storing user information in **Amazon RDS (MySQL)**.

    b. Uploading files to the **Amazon S3** bucket.
9. Every file uploaded to Amazon S3 generates an **ObjectCreated** event, which automatically triggers the **AWS Lambda** function.
10. The Lambda function extracts the uploaded file details and sends an email notification using **Amazon SES**. Execution logs are stored in **Amazon CloudWatch Logs**.
11. **Prometheus** collects application and Kubernetes metrics through ServiceMonitors.
12. **Grafana** visualizes the collected metrics using dashboards for cluster and application monitoring.
13. **Promtail** collects container logs and forwards them to **Loki** for centralized log management.
14. **Alertmanager** generates alerts based on Prometheus alert rules, such as backend availability and HTTP request monitoring, ensuring continuous observability of the application.

6. Technology Stack

Frontend

- HTML
- CSS
- JavaScript
- Nginx
- Docker

Backend

- Python
- Flask
- Gunicorn
- Docker

Cloud Services

- Amazon EKS
- Amazon ECR
- Amazon RDS (MySQL)
- Amazon S3
- AWS Lambda
- Amazon SES
- AWS Secrets Manager

- IAM

Infrastructure as Code

- Terraform

Container Orchestration

- Kubernetes
- Argo Rollouts (Canary Deployment)
- ArgoCD (GitOps)

CI/CD

- GitHub Actions (Continuous Integration)
- ArgoCD (Continuous Deployment)

Monitoring & Logging

- Prometheus
- Grafana
- Loki
- Promtail
- Alertmanager

7. Conclusion

This project successfully implemented a complete **cloud-native DevOps platform** on AWS. Infrastructure provisioning, application deployment, CI/CD automation, GitOps, canary deployments, monitoring, centralized logging, secure secret management, persistent database storage, and serverless event-driven automation were integrated into a single production-ready workflow.

The project demonstrates best practices for deploying scalable, observable, and automated applications using modern DevOps tools and AWS cloud services.